

CHE 565 -- Homework 5

Soki Sem

April 29, 2026

Introduction

The following homework was done with Simulink and the Control Systems Library in Python. The block diagram of the system is shown in Figure 1. The block diagram was created in simulink and then I built the same diagram as a python function using the control library. Simulink was mainly used as a sanity check for the response of the system in the python code.

Note: sum2 in the block diagram is the summing junction that takes the first controller output and subtract the P stream in order to form the inner loop. But, first in order to simulate without the inner loop, so I set cascade=False. This just converts the controller output (Yc1) to the error signal (E2) for the second controller. The disturbance D is added directly to the output of Gp1 (Yp1)

The transport delay transfer function is approximated using a Pade approximation of order 1. Although it should be relatively straightforward to simulate with delay in simulink and in python, I commented through the delay blocks in simulink and in python. The assignment didn't specify the delay time. Also, the control library has the delay function, but it only uses the Pade approximation.

Which gives the following transfer function approximation:

$$e^{-s\theta_d} = \frac{-\frac{s\theta_d}{2} + 1}{\frac{s\theta_d}{2} + 1}$$

```
1 import control as ct
2
3 def build_closed_loop(Kc1, Kc2, tauI1, tauI2, cascade=False):
4
5     s = ct.tf('s')
6     t = sp.symbols('t', real=True)
7     I1 = 1.0 / tauI1
8     I2 = 0
9     numD, denD = ct.delay.pade(theta, pade_order)
10
11     Gc1 = Kc1 * (1 + I1 / s)
12     Gc2 = Kc2 * (1 + I2 / s)
13     Gp1 = Kp1 / (taup1 * s + 1)
14     Gp2 = Kp2 / (taup2 * s + 1)
15     Gd = ct.tf([1], [1]) # direct disturbance addition
16     GD1 = ct.tf(numD, denD, name='GD1', inputs='Yp1', outputs='YD')
17     GD2 = ct.tf(numD, denD, name='GD2', inputs='Yp2', outputs='Y')
18     # State Space Representation of the blocks for interconnection
19     # Note: the control library's interconnect function works better with state-space models
20     # so we convert the transfer functions to state-space form.
21     # xdot = Ax + Bu
22     # y = Cx + Du
23
24     Gc1_blk = ct.ss(Gc1, name='Gc1', inputs='E1', outputs='Yc1')
25     Gc2_blk = ct.ss(Gc2, name='Gc2', inputs='E2', outputs='Yc2')
26     Gp1_blk = ct.ss(Gp1, name='Gp1', inputs='Yc2', outputs='Yp1')
```

```

26 Gp2_blk = ct.ss(Gp2, name='Gp2', inputs='P', outputs='Y')
27 Gd_blk = ct.ss(Gd, name='Gd', inputs='D', outputs='Yd')      # direct disturbance
                        addition
28 #GD1_blk = ct.ss(GD1, name='GD1', inputs='Yp1', outputs='YD')
29 #GD2_blk = ct.ss(GD2, name='GD2', inputs='Yp2', outputs='Y')
30 sum1 = ct.summing_junction(inputs=['Ysp', '-Y'], output='E1', name='Sum1')
31 if cascade:
32     sum2 = ct.summing_junction(inputs=['Yc1', '-P'], output='E2', name='Sum2')
33 else:
34     sum2 = ct.summing_junction(inputs=['Yc1'], output='E2', name='Sum2')
35 sum3 = ct.summing_junction(inputs=['Yp1', 'Yd'], output='P', name='Sum3')
36 control_blks = [Gc1_blk, Gc2_blk]
37 plant_blks = [Gp1_blk, Gp2_blk]
38 disturbance_blks = [Gd_blk]
39 #delay_blks = [GD1_blk, GD2_blk]
40 sum_blks = [sum1, sum2, sum3]
41 blocks = control_blks + plant_blks + disturbance_blks + sum_blks
42 sys = ct.interconnect(
43     blocks,
44     input=['Ysp', 'D'],
45     output=['Y'],
46     input_prefix = ["Ysp", "D"],
47     output_prefix = ["Y"],
48 )
49 return sys
50
51
52 def calculate_IAE(t, y, ysp):
53     error = ysp - y
54     iae = np.trapezoid(np.abs(error), t)
55     return iae
56
57 def simulate_case(sys, t, ysp_input, d_input):
58     U = np.vstack([ysp_input, d_input])
59     resp = ct.forced_response(sys, T=t, U=U, squeeze=True, return_states=True)
60     return resp.time, resp.outputs, resp.inputs, resp.states
%

```

Problem 1

Closed-loop Response to Step Disturbance

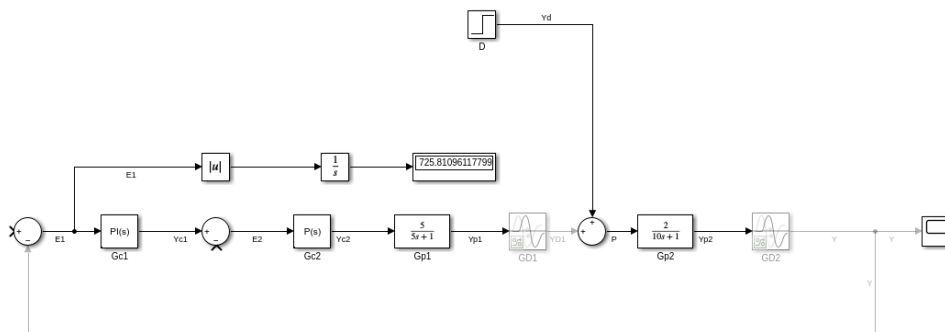


Figure 1: Block diagram of the closed-loop system without cascade control and with a unit step disturbance from Simulink.

The Simulink block diagram is shown in Figure 1.

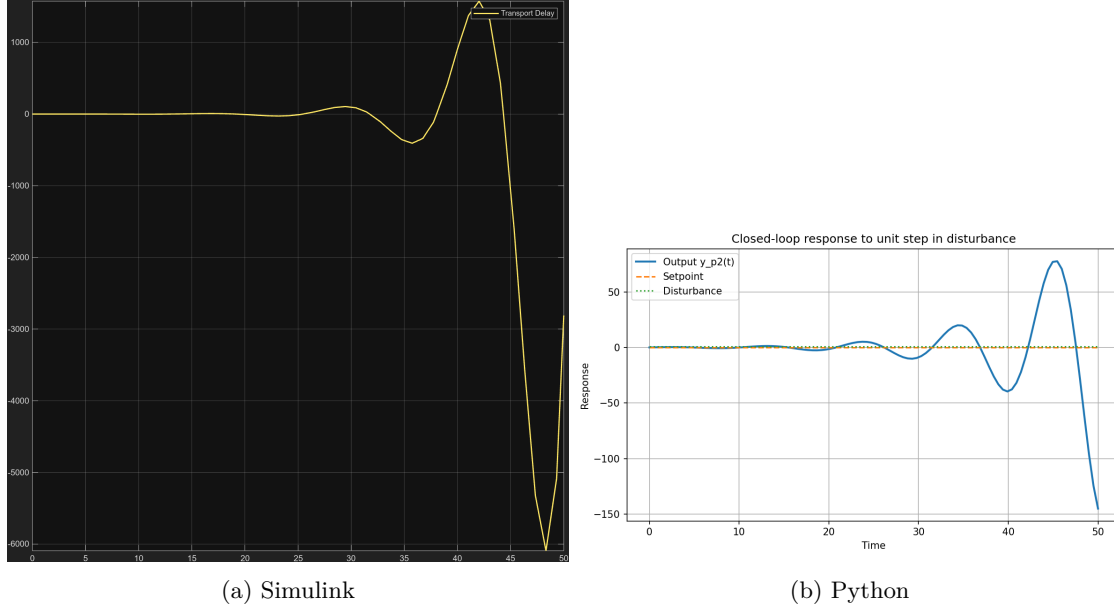


Figure 2: Closed-loop response to no step change in setpoint and a unit step change in disturbance with no cascade control and no autotuning.

Case 1: Step disturbance with no setpoint change gave and step in disturbance with no setpoint change gave $IAE = 725.7464$ in Python and $IAE = 725.8109$ in Simulink. The closed-loop disturbance response and no cascade loop is shown in Figure 2.

The PI controller was then tuned using the autotuning feature in Simulink, which gave $P = 0.1837$ and $I = 0.0816$.

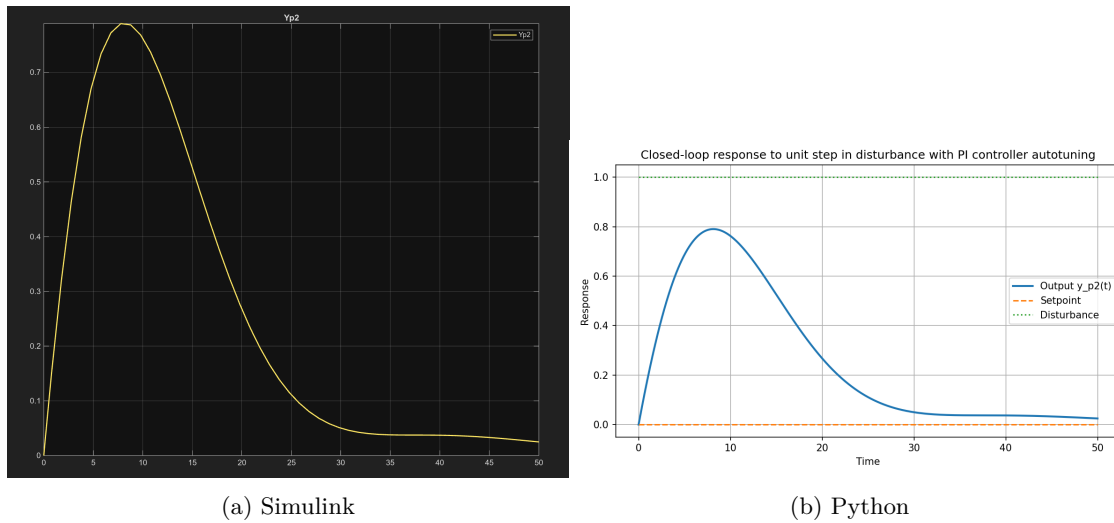


Figure 3: Closed-loop response to no step change in setpoint and a unit step change in disturbance with autotuning.

Case 2: Step disturbance with no setpoint change and PI autotuning gave $IAE = 13.0420$ in Python and $IAE = 13.0463$ in Simulink. The response to a unit step change in disturbance with no cascade control and

PI autotuning is shown in Figure 3.

Closed-loop Response to Step Setpoint Change

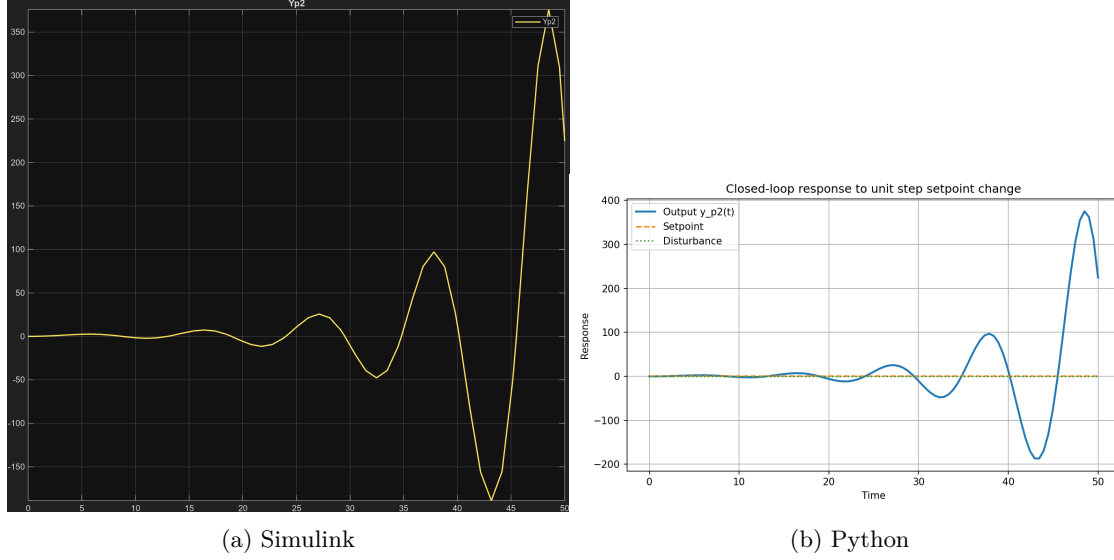


Figure 4: Closed-loop response to step change in setpoint and no step disturbance.

Case 3: Step setpoint change with no disturbance change gave $IAE = 2443.4231$ in Python and $IAE = 2450.8205$ in Simulink. The response is shown in Figure 4.

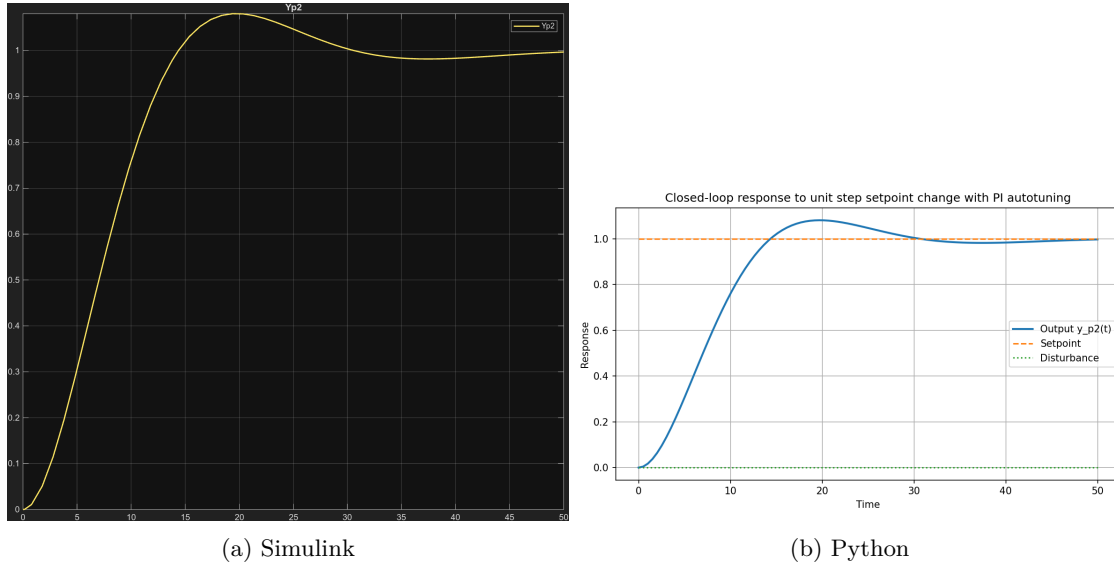


Figure 5: Closed-loop response to step change in setpoint and no step disturbance with PI autotuning.

Case 4: Step setpoint change with no disturbance change and PI autotuning gave $IAE = 8.1947$ in Python and $IAE = 8.1939$ in Simulink. The response is shown in Figure 5.